

class06: r functions

erica yue hu (pid:a17787225)

background

functions are at the heart of using R. Everything we do involves calling and using functions (from data input)

all functions in R have at least 3 things:

1. A **name** the thing we use to call the function.
2. one or more input **arguments** that are comma separated
3. the **body**, lines of code between curly brackets { } that does the work of the function

A first function

lets write a silly wee function to add some numbers:

```
add<- function (x) {  
  x+1  
}
```

lets try it out

```
add(100)
```

```
[1] 101
```

will ts work?

```
add(c(100,200,300))
```

```
[1] 101 201 301
```

modify it to be more useful and add more than just 1

```
add <- function (x,y=1) {  
  x+y  
}
```

^this means that if you don't have y, it is gonna just be 1

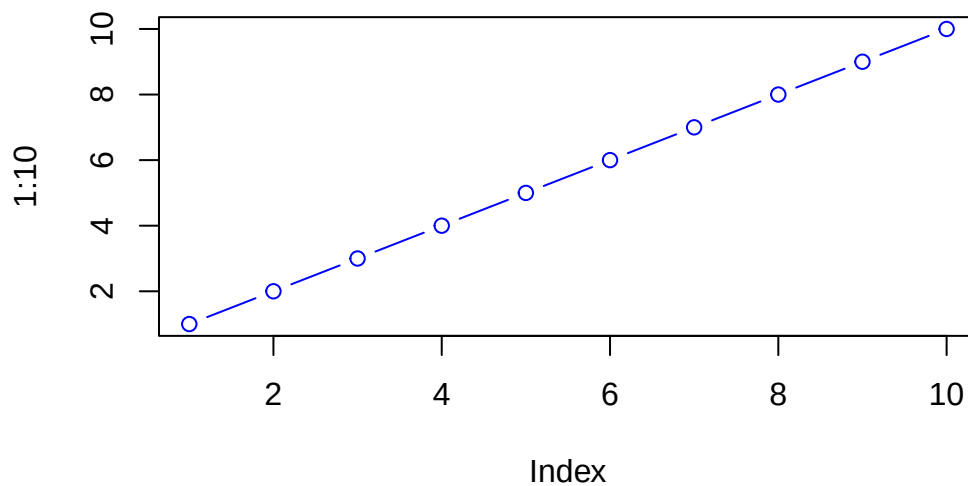
```
add(100,10)
```

```
[1] 110
```

```
add (100)
```

```
[1] 101
```

```
plot(1:10, col="blue",typ="b")
```



```
log(10, base = 10)
```

```
[1] 1
```

N.B Input arguments can be either **required** or **optional**. the later have a fall-back default that is specified in the function code with an equals sign.

```
#add(x=100,y=200,z=300) ts does not work cuz we have notin for z.
```

##A second function all functions in R look like this

```
name <- function (arg) {  
  body  
}
```

the `sample()` function in R takes a sample of the specified size from the elements of `x` using either with or without replacement.

```
sample(1:10, size = 4)
```

```
[1] 7 2 6 9
```

Q. Return 12 numbers picked randomly from the input 1:10

```
sample(1:10, size =12, replace = TRUE)
```

```
[1] 5 7 10 9 9 3 9 1 10 5 2 3
```

`replace = true` allows you to get the same number for more than once

Q. write the code to generate a random 12 nucleotide long DNA sequence?

```
sample( c("A","C","G","T"), size = 12, replace = TRUE)
```

```
[1] "A" "G" "C" "A" "A" "A" "G" "T" "T" "C" "G" "T"
```

another way to write ts:

```
bases <- c("A","C","G","T")  
sample(bases, size=12, replace = TRUE)
```

```
[1] "A" "G" "C" "A" "C" "A" "G" "C" "C" "A" "A" "G"
```

Q. write a first version function called `generate_dna()` that generates a user specified length `n` random DNA sequence?

```
name <- function (arg) {  
  body  
}
```

```
generate_dna <- function(n=6) {  
  bases <- c("A","C","G","T")  
  sample(bases, size = n, replace = TRUE)  
}
```

```
generate_dna(100)
```

```
[1] "G" "C" "G" "G" "T" "T" "T" "G" "G" "A" "G" "G" "C" "C" "T" "C" "C" "C"  
[19] "T" "C" "G" "C" "T" "G" "C" "T" "G" "T" "T" "A" "G" "A" "T" "C" "G" "G"  
[37] "T" "T" "T" "C" "C" "G" "T" "G" "C" "A" "A" "C" "C" "C" "T" "T" "C" "A"  
[55] "G" "A" "G" "T" "G" "A" "C" "T" "G" "A" "T" "T" "T" "T" "T" "T" "T" "A"  
[73] "T" "C" "T" "A" "T" "C" "G" "C" "C" "T" "A" "C" "A" "G" "A" "C" "A" "C"  
[91] "C" "A" "C" "G" "C" "G" "G" "C" "T" "T"
```

```
#this calls the function
```

`n = 6`, just means tht if you don't put anything in, then it is gonna run as 6

Q.modify ur function to return a FASTA like seequence so rather than [1] "G" "C"
"A" "A" "T" we want "GCAAT".

```
generate_dna <- function(n=6) {  
  bases <- c("A","C","G","T")  
  seq=sample(bases, size = n, replace = TRUE)  
  paste(seq, collapse = "")  
}
```

```
generate_dna(10)
```

```
[1] "CTTAAACATC"
```

Q. give the user an option to return FASTA format output sequence or standard multi-element vector format

```
generate_dna <- function(n=6, fasta=TRUE) {  
  bases <- c("A","C","G","T")  
  seq<-sample(bases, size = n, replace = TRUE)  
  
  if(fasta){  
    seq<-paste(seq, collapse="")  
    cat("Hello...")  
  } else {  
    cat ("...is it me ur looking for...")  
  }  
  
  return(seq)  
}
```

```
generate_dna(10,fasta=T)
```

Hello...

```
[1] "TTAAGCAACA"
```

```
generate_dna(10,fasta=F)
```

...is it me ur looking for...

```
[1] "G" "A" "G" "A" "G" "A" "C" "A" "T" "C"
```

if fasta condition is fulfilled, then it is going to do everything in the sequence that is listed inside the if loop

##A new cool function >Q. write a function called **generate_protein()** that generates a user specified length protein sequence in FASTA like format?

Q. use ur new **generate_protein()** function to generate sequences between length 6 and 12 amino acids in length and check any of these are unique in nature (i.e found in the NR database at NCBI)

```
generate_protein<- function(n=10){
  aa <- c("A","R","N","D","C","Q","E","G","H","I",
    "L","K","M","F","P","S","T","W","Y","V",
    "U")
  seq = sample(aa, size=n, replace=TRUE)

  return (paste(seq, collapse=""))
}
```

```
generate_protein(30)
```

```
[1] "TANIAEPHSFDUAIHISEFTDLQVHKKGLK"
```

Question 2: using a for loop

```
generate_protein<- function(n){
  aa <- c("A","R","N","D","C","Q","E","G","H","I",
    "L","K","M","F","P","S","T","W","Y","V",
    "U")
  seq = sample(aa, size=n, replace=TRUE)

  return (paste(seq, collapse=""))
}
```

```
for (i in 6:12) {
  cat(">", i, "\n", sep = "")
  cat (generate_protein(i),"\n")
}
```

```
>6
YCSMFK
>7
WISDSRF
>8
DLRHFFDV
>9
ESTDILYAH
>10
ANGNMANVNI
>11
```

UKNSQYLWNMK
>12
KASWWGCCVVGC